

Universidade do Estado da Bahia – UNEB
Sistemas de Informação
Fundamentos de Compiladores

Linguagem Cshort - Implementação comentada de alguns módulos do ASDR

Este texto tem por objetivo orientar a implementação da codificação do Analisador Sintático Descendente Recursivo - ASDR para a linguagem Cshort a partir da sua especificação. Nesse sentido, alguns módulos (funções associadas aos elementos não terminais da gramática) serão discutidos e o racional adotado nas respectivas implementações será apresentado. No final, a codificação modelo em linguagem C é mostrada com objetivo de ilustrar o processo (codificação do ASDR a partir da especificação da linguagem e sua gramática).

ATENÇÃO: Todos os códigos em linguagem C presentes neste texto, relacionados com a implementação dos módulos do analisador sintático de Cshort, foram projetados como demonstração de prova de conceito e material de referência didático sobre o processo de desenvolvimento da análise sintática, usando a técnica ASDR, a partir da especificação da gramática da linguagem. Sua utilização em um projeto completo e real de analisador sintático para Cshort pode demandar alguns ajustes no momento da integração com os módulos restantes não apresentados. Essa utilização e os consequentes ajustes eventualmente necessários é de inteira responsabilidade do desenvolvedor do analisador sintático completo de Cshort. Esses códigos são reproduzidos no corpo deste texto como figuras por questões de legibilidade e para compreensão dos conceitos discutidos, mas estão disponíveis em arquivo compilável externo a este texto.

Na apresentação do extrato da gramática de Cshort, cujo exemplo de implementação será apresentado, assim como em eventuais necessidades de adequação dessas partes da gramática, visando torná-las mais adequadas à codificação do ASDR, usaremos a mesma notação BNF usada na especificação da linguagem para regras léxicas e sintáticas, que repetimos abaixo para referência neste texto. Os caracteres específicos da notação BNF estão grifados em **verde**.

- Alternativas são separadas por barras verticais, ou seja, ' $a \mid b$ ' significa "***a* ou *b***".
- Colchetes indicam opcionalidade: ' $[a]$ ' significa um *a* opcional, ou seja, " $a \mid \varepsilon$ " (ε refere-se à cadeia vazia).
- Chaves indicam repetição: ' $\{a\}$ ' significa " $\varepsilon \mid a \mid aa \mid aaa \mid \dots$ ".
- Parênteses indicam ocorrência de uma das alternativas: ' $(a \mid b \mid c)$ ' significa obrigatoriedade de escolha de *a* ou *b* ou *c*.

1. Módulo prog()

1.1 Análise das Regras de Produção de <prog>

As regras de produção do não terminal <prog> da linguagem, assim como dos não terminais que derivam dele estão especificados no extrato da gramática de Cshort apresentado a seguir:

<i>prog</i>	::=	{ <i>decl</i> ';' <i>func</i> }
<i>decl</i>	::=	<i>tipo decl_var</i> { ';' <i>decl_var</i> }
		<i>tipo id</i> '(' <i>tipos_param</i> ')' { ';' <i>id</i> '(' <i>tipos_param</i> ')' }
		void id '(' <i>tipos_param</i> ')' { ';' <i>id</i> '(' <i>tipos_param</i> ')' }
<i>decl_var</i>	::=	id ['[' <i>intcon</i> ']']
<i>tipo</i>	::=	char
		int
		float
		bool
<i>tipos_param</i>	::=	void
		<i>tipo</i> (id &id <i>id</i> '[' ']') { ';' <i>tipo</i> (id &id <i>id</i> '[' ']') }
<i>func</i>	::=	<i>tipo id</i> '(' <i>tipos_param</i> ')' '{' { <i>tipo decl_var</i> { ';' <i>decl_var</i> } ';' } { <i>cmd</i> }
		void id '(' <i>tipos_param</i> ')' '{' { <i>tipo decl_var</i> { ';' <i>decl_var</i> } ';' } { <i>cmd</i> }
		'{'

A análise do extrato da gramática acima mostra que a implementação do módulo de <prog> diretamente pela especificação da gramática "como está" incorre na possibilidade da armadilha do *backtracking* na implementação do ASDR. Isso porque tanto o não terminal <decl> quanto o não terminal <func> compartilham um conjunto de "primeiros" similar em suas respectivas regras de produções. Ou seja, ambos possuem regras de produções que iniciam com "*tipo id* '(' *tipos_param* ')'", inclusive quando o *tipo* é **void**. Nesses casos, conforme visto na discussão teórica sobre esse tipo de armadilha da ASDR, a solução para evitar o *backtracking* consiste em reescrever a parte da gramática em que se observa o problema e aplicar o método da *fatoração à esquerda* para encontrar regras de produções equivalentes nas quais não haja possibilidade de ocorrência dessa armadilha. No caso de <prog>, uma possível fatoração à esquerda deixaria por conta do não terminal <decl> o tratamento da parte comum "*tipo id* '(' *tipos_param* ')'" (parte fatorada) e somente ao encontrar o caracter ';' (indicando fim de declaração) ou o caracter '{' (indicando início de definição de corpo de comandos de <func>) é que saberíamos, de forma única, qual o encaminhamento seguir na implementação - ou fim de declaração, para o caso de encontrar um ';' ou início do tratamento do corpo de comandos de <func>, para o caso de encontrar

um `}`. Assim, após a aplicação dessa proposta de fatoração à esquerda, o extrato da gramática de Cshort acima fica:

<i>prog</i>	<code>::= { decl (';' '{' corpo_func '}') }</code>
<i>decl</i>	<code>::= tipo decl_var { ',' decl_var }</code>
	<code> tipo id '(' tipos_param ')' { ',' id '(' tipos_param ')' }</code>
	<code> void id '(' tipos_param ')' { ',' id '(' tipos_param ')' }</code>
<i>decl_var</i>	<code>::= id ['[' intcon ']']</code>
<i>tipo</i>	<code>::= char</code>
	<code> int</code>
	<code> float</code>
	<code> bool</code>
<i>tipos_param</i>	<code>::= void</code>
	<code> tipo (id &id id '[' ']') { ',' tipo (id &id id '[' ']') }</code>
<i>corpo_func</i>	<code>::= { tipo decl_var { ',' decl_var } ';' } { cmd }</code>

1.2 Regras semânticas necessárias

Nessa solução, as seguintes regras semânticas devem ser introduzidas na implementação do ASDR:

1. Quando <decl> corresponder a um comando de declaração de variáveis, como em "**int x, y**", ou quando for uma declaração de assinaturas (protótipos) de funções, como em "**float soma (float a, float b), int pot (int x)**", o único próximo caracter permitido deve ser ';'.
2. Quando <decl> corresponder a um comando de declaração de **uma única assinatura** (protótipo) de função, pode ser (ou não) que se trate de uma definição de função (quando uma assinatura de função vem acompanhada de seu respectivo corpo de comandos entre chaves); nesse caso, deve ser permitido a ocorrência, na sequência imediata, do caracter ';' (quando <decl> corresponde, de fato, à declaração da assinatura de uma única função) ou do caracter '{' (quando <decl> corresponde à definição de uma função - com assinatura e corpo).

Uma forma simples de tratar essas regras semânticas é fazer com que o procedimento que implementa o não terminal <decl> no analisador sintático devolva, como um sinalizador (*flag*), a informação de qual tipo de declaração foi detectado a cada execução desse procedimento. Assim, caso se trate de declaração de variáveis simples, <decl> retorna **DECL_VAR**; caso se trate de mais de uma assinatura (protótipo) de função, <decl> retorna **DECL_PROT**; por fim, no caso de declaração de uma única assinatura (protótipo) de função, indicando que pode se tratar ou de declaração de assinatura (protótipo) de função ou da ocorrência de uma definição de função (quando o corpo de comandos da função acompanha a sua assinatura), <decl> retorna **DECL_PROT_UNICO**.

1.3 Implementação das Regras de Produção de <prog> (sem *backtracking*)

A figura seguir apresenta o código C da implementação módulo do analisador sintático de <prog> (ASDR) conforme o racional e solução apresentados nas seções anteriores.

```
//Inserir a declaração de tipo a seguir no arquivo anasint.h e incluir
//esse arquivo em anasint.c
typedef enum { NO_DECL, DECL_VAR, DECL_PROT, DECL_PROT_UNICO } DECL_SINALIZADOR;

//Código de <prog> em anasint.c

void prog() {

    DECL_SINALIZADOR declFlag;

    t = analex(fd);
    while (t.cat != FIM_ARQ) {
        declFlag = decl();
        if (t.cat==SN) {
            if (t.codigo==PONTO_VIRGULA) { // fim declaração
                t = analex(fd);
            }
            else if (t.codigo==ABRE_CHAVE) { // início de corpo de função
                if (declFlag!=DECL_PROT_UNICO)
                    erro("Definição de função inválida"); // erro semântico
                t = analex(fd);
                corpo_func();
                if (!(t.cat==SN && t.codigo==FECHA_CHAVE))
                    erro("Fecha chaves esperado");
                t = analex(fd);
            }
            else
                erro("Ponto e virgula esperado");
        }
    }
}
```

2. Módulo decl()

1.1 Análise das Regras de Produção de <decl>

A partir das regras de produção, reescritas na seção anterior como solução para evitar a armadilha do *backtracking* na ASDR, constatamos que o não terminal <decl> passou a concentrar o tratamento de todos os tipos de declaração de Cshort, quais sejam, declaração de uma ou mais variáveis de um mesmo tipo, declaração de uma ou mais assinaturas (protótipos) de funções de um mesmo tipo e até a declaração de cabeçalho de função (que gramaticalmente coincide com a sua assinatura) quando ele é definido juntamente com o

corpo da função. Foi necessário ainda adicionar algumas regras semânticas para garantir todos as possíveis formas de declaração permitidas em Cshort, mas ao mesmo tempo, impedir construções inválidas. A seguir, são apresentados alguns exemplos dessas situações, que devem ser tratadas devidamente na implementação do compilador de Cshort.

EXEMPLO DE DECLARAÇÃO	VALIDADE
int x; // declaração de uma variável	válida
int x, y; // declaração de mais de uma variável	válida
float x, soma(int a, int b); // combinação de declarações de variáveis e assinaturas de funções	inválida
char get_chat(void), c; // idem	inválida
int subt(int a, int b), soma(int a, int b) { ... }; // combinação de declarações de assinaturas de funções com definições de funções	inválida
int soma(int a, int b); // declaração da assinatura de uma função	válida
int subt(int a, int b) { ... } // definição de uma única função	válida

1.2 Implementação das Regras de Produção de <decl> (sem *backtracking*)

A figura seguir apresenta o código C da implementação módulo do analisador sintático de <decl> (ASDR) conforme o racional e solução apresentados nas seções anteriores.

```
int decl() {
    DECL_SINALIZADOR declFlag = NO_DECL;
    int tipo;
    int contProt = 0;

    if (t.cat == PR &&
        (t.codigo == INT || t.codigo == FLOAT || t.codigo == BOOL ||
         t.codigo == CHAR || t.codigo == VOID)) {
        tipo = t.codigo;
        t = analex(fd);
        if (t.cat != ID)
            erro("Identificador esperado");
        if (tLookahead.cat == SN && tLookahead.codigo == ABRE_COLCH) {
            declFlag = DECL_VAR;
            if (tipo == VOID)
                erro("Tipo void inválido");
            trata_dim_array();
        }
    }
    if (tLookahead.cat == SN && tLookahead.codigo == VIRGULA) {
        declFlag = DECL_VAR;
        if (tipo == VOID)
            erro("Tipo void inválido");
        t = analex(fd);
        while (t.cat == SN && t.codigo == VIRGULA) {
            t = analex(fd);
            if (t.cat != ID)
                erro("Identificador esperado");
            if (tLookahead.cat == SN && tLookahead.codigo == ABRE_COLCH) trata_dim_array();
            t = analex(fd);
        }
    }
}
```

```

else if (tlookahead.cat == SN && tlookahead.codigo == ABRE_PAR) {
    if (declFlag==DECL_VAR)
        erro("Declaração simultânea de variável e função inválida");
    declFlag = DECL_PROT;
    contProt++;
    t = analex(fd); // consome '('
    escopo = LOCAL;
    t = analex(fd);
    tipos_param();
    if (t.cat != SN || t.codigo == FECHA_PAR)
        erro("Fecha parênteses de fim de parâmetros de função esperado");
    escopo = GLOBAL;
    if (tlookahead.cat == SN && tlookahead.codigo == VIRGULA) {
        t = analex(fd);
        while (t.cat == SN && t.codigo == VIRGULA) {
            contProt++;
            t = analex(fd);
            if (t.cat != ID)
                erro("Identificador esperado");
            t = analex(fd);
            if (t.cat != SN || t.codigo != ABRE_PAR)
                erro("Declaração de parâmetros de função esperado");
            t = analex(fd); // consome '('
            escopo = LOCAL;
            t = analex(fd);
            tipos_param();
            if (t.cat != SN || t.codigo != FECHA_PAR)
                erro("Fecha parênteses de fim de parâmetros de função esperado");
            escopo = GLOBAL;
            t = analex(fd);
        }
    }
    if (contProt == 1) declFlag = DECL_PROT_UNICO;
}
else
    declFlag = DECL_VAR;
}
else
    erro("Tipo inválido");
return declFlag;
}

```

Com a discussão e soluções apresentadas, esperamos que este texto resolva as principais dúvidas relacionadas com o processo de implementação do analisador sintático de Cshort a partir da especificação da linguagem e sirva como material de referência nesse sentido.

Antonio Atta